Migrating from Terraform Community Edition (CE/OSS) to Terraform Enterprise (TFE) involves several specific code and workflow changes, especially around backend configuration, state management, and automation. Here's a detailed breakdown of what you need to know and what code changes are typically required—especially if you want to automate this process (with AI or otherwise). This version includes narrative on how Morgan Stanley is currently approaching this.

Morgan Stanley Note:
The current approach that MS is planning and already working on is to use the existing overarching cloud-deploy pipeline to manage the CE to TFE migration activities. The current Terraform usage, at least for the legacy TFE and for some or all of the CE Terraform operations uses specialized processes in cloud-deploy currently and is intended to be modified as needed to support the new platform. Consumer onboarding is part of this flow such as creating Workspaces and other artifacts in TFE, managing all the standards and naming conventions, etc..

They are then planning to execute the migrations using the same tool. In a specific workflow it would accomplish most of what is described in starting in step 2 below. The consumer would point to their current source code and state file somehow and provide any other project info not derivable already and it will migrate the control of that infrastructure into TFE. This would involve creating Projects, Workspaces, variable sets, and probably other artifacts like teams and users (some of this may be an administrative task outside cloud-deploy), and migrating the Terraform state to the workspace. Possibly perform the first plan run, or the consumer would know to do that next. Then if it is clean they could apply no changes or wait for the next need to make changes. All future runs would then happen in TFE.

They intend to migrate as-is with no modifications to meet new compliance or other standards. In theory they have been operating with some compliance adherence, but not sure how much and how. It is possible cloud-deploy does some checking before or after a run today. With HC/IBM guidance, they are building a pre-migration validation step that would help ensure the migration would be successful and meets some requirements. A key check is to ensure that the versions of Terraform CLI and Providers are at least at a minimum level. If not, the consumer would be required to upgrade to meet requirements before continuing. This by itself is a vulnerability avoidance, as earlier versions certainly have corrected CVEs etc..

So with the current plan, the main opportunity to introduce Bob into the migration would the code change in step 1. It is possible that an argument could be made that the complete preparation for migration could/should be done on the consumer side, where Bob as a coding assistant could not only make the minimal code changes, but also perform the validation step, then possibly the corrective actions like upgrading versions and attempting runs and remediating identified discrepancies. Possibly even performing the remainder of the onboarding tasks using TFE APIs etc., but that may be a stretch as it likely repeats and must model logic already in cloud-deploy. But it could execute the cloud-deploy step and analyze/remediate those outcomes.

One other consideration would be around if there is infrastructure not yet under Terraform control. Don't know, but certainly likely given the vast CE landscape. There are facilities to import cloud resources into TF control, but involves detailed discovery and coding of not only the Terraform code that would manage it going forward, but also the import blocks/mechansims to assume initial control. One area that could be explored as it is a good AI use case.

For the last item and most of this discussion, it is important to note that there is a lot of development and product roadmap items around the injection of AI into Terraform processes to accomplish the automation of exactly these kinds of actvities. This could be mentioned in early reccomendations and not neccessarily included in early demos, and then fully evaluated and incorporated in the real final design.

Extra Bob Scenarios

A concern is that MS may not want Bob to handle functionality already designated to cloud-deploy like Workspace creation and terraform runs. With that assumption, it still could be possible for Bob to execute cloud-deploy activities, analyze results, and remediate terraform runs and related code.

Our understanding is that cloud-deploy is a CLI driven workflow that has the capability to integrate with many tools and can ensure compliance and standards to some level. Authentication to such tools, CSPs and environments must be handled. Bob should be able to be trained how to initiate cloud-deploy activities and interpret the output that a human would need to review and react to.

MS has another older TFE environment as well as a lot of CE Terraform provisioning. It is our understanding/assumption that cloud-deploy handles the existing TFE environment activities and that the platform team assumes it will be used to do the same for the new TFE environment. It is most likely that Bob uses TFE APIs to handle any activities that need to configure or run on TFE.

Migration of App-A infrastucture from CE to TFE Outline:

- Consumer identifies state location and code location
- Bob examines code (and maybe state) and determines changes required:
    - Terraform version age - need upgrade?
    - Provider(s) in allowed list?
    - Provider(s) version age - need upgrade?
    - backend block needs change for cloud block used in TFE
    - Other code/state best practices being violated (secrets exist in code or state)
- Bob reports results and decision if migration should proceed
- Bob makes code adjustments (in-place in local directory or manages brannching, PR, etc..)
- Bob executes cloud-deploy for TFE Workspace and other artifact creation
- Together with that or separately Bob executes cloud-deploy for Terraform Plan

- Bob evaluates Plan output
    - Success
        - Bob executes cloud-deploy for Terraform Apply
        - Bob evaluates Apply output
            - Success - migration complete!
        - Failure
            - Bob iterates corrections and retry of Apply until Success
    - Failure
        - Bob iterates corrections and retry of Plan until Success
- Bob documents results and marks migration done

Related Scenarios

- All tests pass, migration goes without event
- CLI or Provider(s) need upgrading
    - Clean migration after upgrade
    - Plan fails due to incompatibilies
        - Bob has to inteliigently determine reason for failure and proper remediation
        - (this can be a difficult manual effort)
        - Outcome hopefully successful remediation, could be Bob reports and fails migration
- Best practice violation
    - Examples, secrets in code or state, hard coded values that should be variables, etc.
        - Bob either remediates based on rules/proper handling or reports error and fails migration

Other Considerations

- When remediation fails, Bob could be trained on proper remediation and apply to future attempts.
- Bob may be able to determine exactly what inputs and other handling a human would need to know and do to execute the Terraform runs in general and the migration request. Resulting in the consumer just gives a Bob a list of repos needing migration and it can operate effectively unattended.
- Infrastructure being migrated would exist in an environment hierachy (dev/test/prod) so Bob should be able to repeat remediations up the chain once the lower environment has been completed.
- Another extension to this could be that Bob does some code refactoring to ensure certain coding or platform standards are met and/or improved upon.
- Assuming that real access and understanding of code-deploy will be difficult to obtain for purposes of a demo, a fairly simple CLI utility could be developed that mimmic the functionality.

- There is a CLI utility developed by a HashiCorp TF expert that front-ends TFE APIs called TFX that could be used to generate such a script. Or regular API calls could be embedded as well.
- HCP Terraform is often used as a demo environment for TFE (or HCPT) scenarios as it is functionality equivalent.

1. Backend Block Changes

From Local/OSS to TFE:

- In CE, state is often managed locally or in a remote backend like S3, Azure Blob, or GCS.
- In TFE, state is managed by the TFE backend, which requires a specific backend block.

Typical Code Change Example:

Before (OSS/CE, using local or S3 backend):

```
terraform {
  backend "local" {
    path = "terraform.tfstate"
  }
}
```

or

```
terraform {
  backend "s3" {
    bucket = "my-bucket"
    key    = "path/to/my/key"
    region = "us-east-1"
  }
}
```

After (TFE, using remote backend):

```
terraform {
  backend "remote" {
    hostname     = "tfe.example.com" # or app.terraform.io for HCP
Terraform
    organization = "my-org"
    workspaces {
      name = "my-workspace"
```

```
    }
  }
}
```

- For Terraform v1.1 and later, you can use the `cloud` block as a more modern alternative:

```
terraform {
  cloud {
    organization = "my-org"
    workspaces {
      name = "my-workspace"
    }
  }
}
```

Automation Note:
If automating, you can script the replacement of backend blocks using tools like `sed`, `yq`, or even AI-driven code refactoring tools. For large-scale migrations, consider using the TFE or HCP Terraform API to create workspaces and upload state programmatically.

Morgan Stanley Note:
This is the part that MS may not have a solution for yet. This code is owned by the future consumers of the TFE Platform, so they cannot orchestrate the changes. They are hoping to find a way to fool the system so changes are not needed to ease the burden on the consumers, but that is not a good idea for long term. Providing the consumers with an automated code conversion may be the best path, but it would require an intelligence to know how the consumer projects will migrate under the Platform Team's control (workspace name, etc.).

This is where a possible alternative is to not migrate through cloud-deploy and let the consumer establish their own precence in TFE and just let the actual Terraform runs be handled by cloud-deploy.

---

2. State File Migration

- You must migrate your existing state files to TFE. This is typically done using the `terraform state push` command or via the TFE API.
- Automation can be achieved by scripting the state upload process, iterating over all your workspaces/modules.

---

3. Workspace Management

- In TFE, each workspace manages its own state. You may need to create corresponding workspaces for each environment/module.
- Use the TFE API to automate workspace creation and configuration.

---

4. Variable and Secret Management

- Variables and secrets are managed in the TFE UI or via the API, not in `.tfvars` files or environment variables as commonly done in OSS. (Often tfvars files are still used in TFE)
- Automation: Use the TFE API to upload variables and secrets in bulk.

---

5. Workflow Changes

- In OSS, you run `terraform plan/apply` locally or in CI/CD.
- In TFE, runs are triggered via the UI, VCS integration, or API. You may need to refactor your automation to use TFE's run triggers.

Morgan Stanley Note:
This is handled by cloud-deploy modifications.

---

6. Automation/AI Considerations

- For AI-driven automation, you can:
  - Parse and update backend blocks in all `.tf` files.
  - Use the TFE API to create workspaces, upload state, and set variables.
  - Script the migration process end-to-end, including validation steps (e.g., running `terraform plan` in TFE after migration).

---

7. Example Automation Flow

1. **Discover all Terraform directories and backend blocks.**
2. **Replace backend blocks** with TFE-compatible configuration.
3. **Create TFE workspaces** via API.
4. **Upload state files** to the corresponding TFE workspaces.
5. **Set variables/secrets** via API.
6. **Trigger a plan** in TFE to validate migration.

---

8. Professional Services

HashiCorp Professional Services offers migration packages and Statements of Work (SOW) to help automate and accelerate this process, ensuring best practices and minimal disruption. This

includes technical planning, workspace mapping, state migration, and post-migration validation.

---

9. Key References & Resources

- Official Migration Guide: Migrating to Terraform Enterprise
- Automating State Migration with the API (Python Example)
- TFE API Docs
- [Professional Services: Terraform Enterprise Migration SOW](No approved Professional Services link is currently available from tool:Retrieve_PSO)

Summary:
Migrating from CE to TFE is primarily about backend block changes, state migration, workspace/variable management, and workflow refactoring. Automation is highly feasible and recommended, especially for large environments. For a seamless, risk-mitigated migration, leverage HashiCorp Professional Services and validated design patterns.